

IT PROFESSIONAL PORTFOLIO

Cloud Engineering and Security Projects

PROJECT 37

Explore Access Key Exposure and Static Credential Risk

AWS IAM | Access Keys | AWS CLI Profiles | EC2 | Session Manager

AUTHOR	Michael Salazar
ENVIRONMENT	Personal AWS Lab
VERSION	v1.0.0
RELEASE DATE	July 28, 2026
STATUS	Complete

Executive Summary

Logged into **TestAccount1** with full administrator access, created the **AKIA** CloudFormation stack from the provided lab template, disabled Session Manager S3 logging because the previous logging bucket was no longer available, and explored the risk of static AWS access keys. The lab first created a temporary IAM user named **deleteme** with no permissions, generated an access key to understand the creation flow, then deactivated the key and deleted the user. The lab then connected to the AKIA EC2 instance through Session Manager, found a stored AWS CLI profile named **SLAW** in the **.aws** directory, viewed the config and credentials files, and used the exposed profile credentials to successfully call **aws ec2 describe-image-attribute**. Running the same command without the profile fell back to the instance role and failed with **UnauthorizedOperation**. The AKIA stack was deleted at the end of the lab.

SECURITY OUTCOME

The security lesson is that static access keys in files can be stolen and used from tooling such as the AWS CLI. The permissions attached to the key matter more than the machine where the key was found.

Business Problem

The previous labs focused on private access paths, VPC endpoints, and Session Manager logging. This lab moves back into identity risk by showing how access keys can sit inside files and be used by command-line tooling. Access keys are long-lived static credentials. If they are embedded in code, images, configuration files, or user home directories, an attacker can extract them and make AWS API calls using whatever permissions those credentials have.

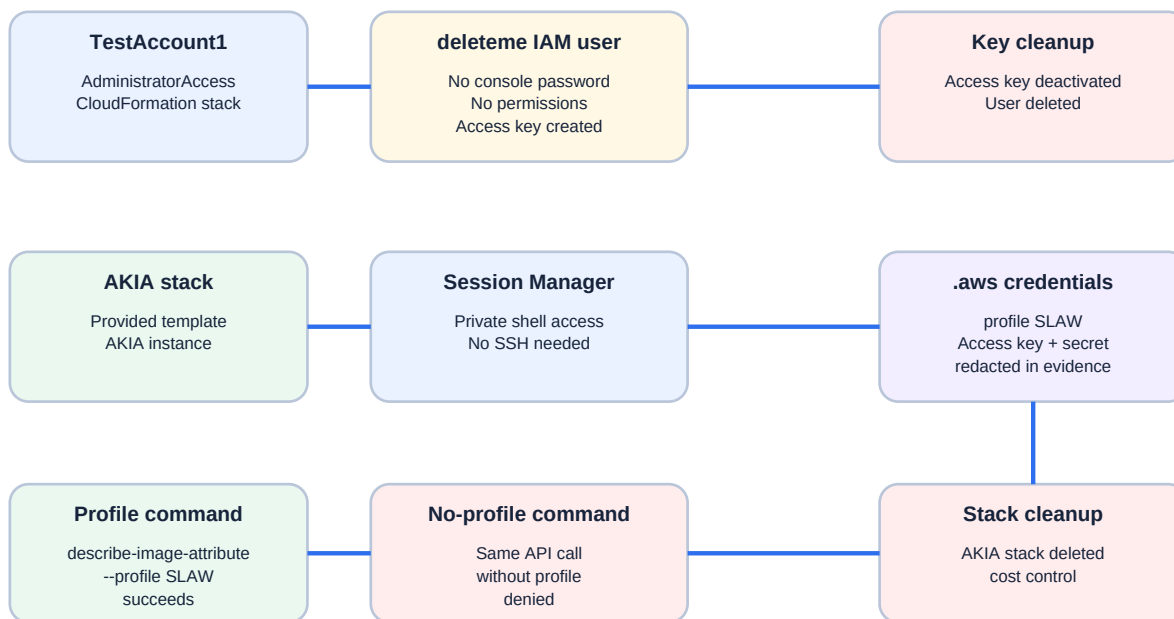
Objectives

- Create the AKIA CloudFormation stack using the provided template URL.
- Disable Session Manager S3 logging because the previous session-manager-logs bucket was no longer available.
- Create a temporary IAM user named deleteme with no console password and no permissions.
- Create an access key for deleteme to observe the access key and secret access key workflow.
- Deactivate the created access key and delete the deleteme IAM user.
- Connect to the AKIA EC2 instance using Session Manager.
- Use the command line to change to the home directory and inspect hidden directories with `ls -la`.
- Enter the **.aws** directory and view the config and credentials files.
- Identify a stored SLAW profile containing an access key and secret access key in the credentials file.
- Use the SLAW profile to call `ec2:DescribeImageAttribute` against a lab AMI and retrieve the expected description.
- Run the same command without the SLAW profile and confirm the instance role is not authorized.
- Delete the AKIA CloudFormation stack after validation.

Environment

Environment	Configuration
Cloud Provider	Amazon Web Services (AWS)
Primary Services	AWS IAM, AWS CloudFormation, Amazon EC2, AWS Systems Manager Session Manager, AWS CLI
Account Context	TestAccount1 / AdministratorAccess
Stack Name	AKIA
Temporary IAM User	deleteme
Temporary User Permissions	No policies attached; no console password
Access Key Handling	Created to view workflow, then deactivated and deleted with user
Instance Access	Session Manager shell into AKIA instance
Credential Location	~/.aws/config and ~/.aws/credentials
Profile Observed	SLAW
Successful Test Command	<code>aws ec2 describe-image-attribute --image-id ami-09fee31bea43efcec --attribute description --profile SLAW</code>
Control Test	Same command without --profile returned UnauthorizedOperation
Cleanup	AKIA CloudFormation stack deleted after lab

Architecture



Static credentials stored in files can override the instance role when a named AWS CLI profile is used.

Figure 1 - Access key creation, discovery, profile use, denial without profile, and cleanup workflow.

Implementation Summary

1. Signed into TestAccount1 with full administrator access.
2. Created the AKIA CloudFormation stack using the provided lab template URL.
3. Disabled Session Manager S3 logging because the prior S3 log bucket was no longer available.
4. Created the IAM user delete IAM user with no console password and no permissions attached.
5. Generated an access key for delete IAM user to view the access key and secret access key workflow.
6. Reviewed AWS access key best-practice warnings, then deactivated the key and deleted the delete IAM user.
7. Connected to the AKIA instance through Session Manager.
8. Changed to the home directory with `cd ~` and listed hidden files with `ls -a`.
9. Entered the `.aws` directory and viewed config and credentials using `cat`.
10. Observed the SLAW profile in config and the SLAW access key and secret key in credentials.
11. Used the SLAW profile to run `describe-image-attribute` against `ami-09fee31bea43efcec` and received the lab completion description.
12. Repeated the command without `--profile` and received `UnauthorizedOperation`, proving the instance role lacked that permission.
13. Deleted the AKIA CloudFormation stack after validation.

Command Validation

```
cd ~
ls
ls -a
cd .aws
ls
cat config
cat credentials
aws ec2 describe-image-attribute --image-id ami-09fee31bea43efcec --attribute description --profile SLAW
aws ec2 describe-image-attribute --image-id ami-09fee31bea43efcec --attribute description
```

The profile-qualified command succeeded because the AWS CLI used the static credentials associated with the SLAW profile. The same command without **--profile SLAW** failed because it used the instance role instead.

Evidence

Figure 2 - Temporary IAM User Created with No Permissions

The screenshot shows the 'Review and create' step of the AWS IAM console. The 'User details' section shows the user name 'deleteme', console password type 'None', and 'Require password reset' set to 'No'. The 'Permissions summary' section shows 'No resources' under the 'Name' column. The 'Tags' section is optional and shows 'No tags associated with the resource'. At the bottom right, there are buttons for 'Cancel', 'Previous', and 'Create user'.

User details		
User name deleteme	Console password type None	Require password reset No

Permissions summary		
Name	Type	Used as
No resources		

Tags - optional
Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.
No tags associated with the resource.
[Add new tag](#)
You can add up to 50 more tags.

[Cancel](#) [Previous](#) [Create user](#)

The IAM evidence shows the *deleteme* user review screen with console password type set to *None*, require password reset set to *No*, and a permissions summary showing *No resources*. This confirms the temporary practice user was intentionally created without permissions.

Figure 3 - Access Key and Secret Access Key Workflow Reviewed

Retrieve access keys [Info](#)

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

AKIA

Secret access key

Q

[Hide](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#)

[Done](#)

The access key evidence shows the Retrieve access keys page with the access key and secret access key redacted. The page also displays AWS best-practice warnings, including not storing access keys in plaintext or code, disabling/deleting keys when no longer needed, using least privilege, and rotating keys regularly.

Figure 4 - Stored AWS CLI Credentials Used by Profile and Denied Without Profile

```
sh-5.29 od -
sh-5.29 ls
sh-5.29 ls -a
ls: ., .aws, .bash_logout, .bash_profile, .bashrc, .lesshst, .ssh
sh-5.29 cd .aws
sh-5.29 ls
config
credentials
sh-5.29 cat config
[profile SLAW]
region = us-west-2
output = json
sh-5.29 cat credentials
[SLAW]
aws_access_key_id = AKIA
aws_secret_access_key =
sh-5.29 aws ec2 describe-image-attribute --image-id ami- --attribute description --profile SLAW
{
  "ImageId": "ami-",
  "Description": {
    "Value": "If you see this you completed the AKIA lab!"
  }
}
sh-5.29 aws ec2 describe-image-attribute --image-id ami- --attribute description
An error occurred (UnauthorizedOperation) when calling the DescribeImageAttribute operation: You are not authorized to perform this operation. User: arn:aws:iam:::assumed-role/SSMClient/... not authorized to
perform ec2:DescribeImageAttribute on resource arn:aws:ec2:us-west-2::image/ami- because no identity-based policy allows the ec2:DescribeImageAttribute action. Encoded authorization failure message: SPMw...
sh-5.29
```

The CLI evidence shows navigation to the home directory, discovery of the hidden .aws directory, config and credentials file review, use of the SLAW profile to retrieve the AMI description, and a failed UnauthorizedOperation when the same API call is run without the SLAW profile. Access keys, secret keys, ARNs, and identifiers are redacted.

Security Analysis

The temporary delete IAM user was safe for the creation exercise because it had no attached permissions and no console password. The purpose was to understand the access key creation flow, not to grant usable access.

The high-risk pattern was the stored SLAW profile inside the .aws directory. A static access key and secret key in a credentials file can be copied and used by the AWS CLI or other SDK tooling.

The successful command with --profile SLAW shows that named profiles can override the instance role path. The CLI used the static profile credentials instead of the role assigned to the EC2 instance.

The failed command without --profile is a useful control. It proves the instance role itself did not have permission for ec2:DescribeImageAttribute on the target image, while the stored profile did.

The risk is not limited to EC2 instances. Static credentials can also be exposed in repositories, laptops, containers, build systems, AMIs, and configuration files.

Deleting the AKIA stack removed the lab instance and reduced the chance of leaving a credential-exposure training environment active.

Lessons Learned

Access keys are static credentials for AWS API access.

AKIA prefixes identify IAM-user-style access keys, while temporary role credentials use a different pattern.

AWS CLI profiles are stored in configuration and credential files under `.aws` in a user home directory.

The credentials file is sensitive because it can hold access key IDs and secret access keys in plaintext.

Using `--profile` tells the AWS CLI exactly which stored credentials to use.

A missing `--profile` option can change the identity used for the API call, usually falling back to the instance role or default profile.

Static credentials should be hunted, rotated, deactivated, or removed wherever possible.

Production Improvements

- Prefer IAM roles, IAM Identity Center, workload identity, or temporary credentials instead of IAM user access keys.
- Use IAM Access Analyzer and credential reports to find IAM users with access keys.
- Monitor CloudTrail for access-key use, especially from unexpected IPs, geographies, or user agents.
- Use Amazon GuardDuty findings for anomalous credential use and exposed credential behavior.
- Scan repositories, AMIs, containers, and CI/CD systems for access keys before deployment.
- Block or alert on long-lived unused access keys and keys older than the rotation policy.
- Use least privilege and permission boundaries where static credentials are temporarily unavoidable.
- Create documented emergency procedures for deactivating exposed keys and reviewing subsequent activity.

Skills Demonstrated

IAM users	Access keys	Secret access keys
Session Manager	AWS CLI profiles	.aws config files
Least privilege	UnauthorizedOperation validation	CloudFormation cleanup

Resume Bullet

RESUME

Analyzed AWS access key exposure risk by creating and removing a no-permission IAM user access key, locating static AWS CLI credentials on an EC2 instance, using a named profile to call an AWS API, validating denial without the exposed profile, and deleting the lab stack after testing.

STAR Interview Story

Situation: The portfolio had already covered private compute administration, but needed a lab showing why long-lived access keys remain a major AWS security risk.

Task: Create a safe access key example, then demonstrate how exposed credentials in an instance profile directory can change AWS CLI authorization behavior.

Action: I created the AKIA stack, disabled stale S3 session logging, created and deleted a no-permission deleteme IAM user and access key, connected to the AKIA instance, found the .aws config and credentials files, used the SLAW profile to call describe-image-attribute, confirmed denial without the profile, and deleted the stack.

Result: The lab demonstrated how static credentials can be exposed in files and used independently of the EC2 instance role, while preserving least-privilege testing and cleanup discipline.

Version History

v1.0.0	July 28, 2026	Initial documented implementation. Created AKIA stack, disabled S3 Session Manager logging, created/deactivated/deleted a no-permission deleteme access key, discovered stored .aws profile credentials, validated SLAW profile API access and no-profile denial, deleted stack, and embedded provided evidence using portfolio documentation standard v1.0.1.

References

- Cloud Security Lab a Week (S.L.A.W.) - Hack My Account with an Access Key/AKIA.
- User-provided IAM deleteme user evidence screenshot.
- User-provided access key retrieval evidence screenshot.
- User-provided AWS CLI credential discovery and validation screenshot.
- Project 35 - Enable Session Manager Logging to S3.
- Project 36 - Deploy a Controlled Public SSH Exposure for GuardDuty Detection.